Exercise 3a

Phase 3 -- Multiple interacting objects

Most real OOP begins here.

Exercise 3: Communication chain

Source, Encoder, Channel, Decoder, Receiver

```
import numpy as np
rng = np.random.default_rng()
```

Source Class

Create:

```
class Source:
```

Internal state:

```
text
LSBfirst
```

Methods:

```
set_text()
set_LSBfirst()
generate()
```

```
class Source:

    def __init__(self, text, LSBfirst=1):
        if len(text) < 1:
            raise ValueError("text must have length > 1")
        self.text = text
        self.LSBfirst = LSBfirst

    def set_text(self, text):
        self.text = text

    def set_LSBfirst(self, LSBfirst):
        self.LSBfirst = LSBfirst

    def generate(self):
        bitsperchar = 8
        textnum = np.array([ord(c) for c in self.text])  # convert ASCII to numeric
        if self.LSBfirst == 1:
            p2 = np.power(2.0, np.arange(0, -bitsperchar, -1))
        else:
            p2 = np.power(2.0, 1 + np.arange(-bitsperchar, 0))
        B = np.array(np.mod(np.array(np.floor(np.outer(textnum, p2)), int), 2), np.int8)
        bits = np.reshape(B, B.size)
        return bits


src = Source("Hi")
bits = src.generate()
print(src.LSBfirst)
print(bits)
```

```
1
[0 0 0 1 0 0 1 0 1 0 0 1 0 1 1 0]
```

```
src.set_LSBfirst(0)
bits = src.generate()
print(src.LSBfirst)
print(bits)
```

```
0
[0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 1]
```

```
#src.set_text("Bye!")
#src.set_LSBfirst(1)
#bits = src.generate()
#print(src.LSBfirst)
#print(bits)
```

Receiver Class

Create:

```
class Receiver:
```

Internal state:

```
noisy
decoded
```

Methods:

```
BERchan()
BERinfo()
received_text()
```

```
class Receiver:

    def __init__(self, noisy, decoded):
        self.noisy = noisy
        self.decoded = decoded

    def BERchan(self, bits):    # channel bit error rate (fraction of flipped noisy bits)
        L = min(len(bits), len(self.noisy))
        BER = np.where(bits[:L]!=self.noisy[:L])[0].size/L
        return BER

    def BERinfo(self, bits):    # information bit error rate (fraction of incorrectly decoded bits)
        L = min(len(bits), len(self.decoded))
        BER = np.where(bits[:L]!=self.decoded[:L])[0].size/L
        return BER

    def received_text(self, LSBfirst=1):
        bitsperchar = 8
        dnb = self.decoded[0:bitsperchar*int(len(self.decoded)/bitsperchar)]
                        # make multiple of bitsperchar long
        B = np.array(np.reshape(dnb, (int(len(dnb)/bitsperchar), bitsperchar)), int)
        if LSBfirst == 1:
            p2 = np.power(2, np.arange(0, bitsperchar))
        else:
            p2 = np.power(2, np.arange(bitsperchar, 0, -1) - 1)
        textnum = np.array(np.dot(B, p2))
        #rxtxt = ''.join(chr(n) for n in textnum)
        rxtxt = ''.join(chr(n%128) for n in textnum)   # convert to 7-bit ASCII
        return rxtxt
```

```
decoded = src.generate()
decoded[3] = np.mod(1+decoded[3],2)
rx = Receiver(decoded, decoded)
```

```
print(rx.BERchan(src.generate()))
print(rx.BERinfo(src.generate()))
print(rx.received_text(src.LSBfirst))
```

```
0.0625
0.0625
Xi
```

Channel Class

Create:

```
class Channel:
```

Internal state:

```
PbE    # Probability of bit error
```

Methods:

```
transmit()
get_PbE()
set_PbE()
```

```
class Channel:

    def __init__(self, PbE):
        if (PbE < 0) or (PbE > 1):
            raise ValueError("PbE must be in range 0...1.0")
        self.PbE = PbE

    def get_PbE(self):
        return self.PbE

    def set_PbE(self, PbE):
        if (PbE < 0) or (PbE > 1):
            raise ValueError("PbE must be in range 0...1.0")
        self.PbE = PbE

    def transmit(self, binary_in):
        L = len(binary_in)
        ix = np.where(rng.random(L) <= self.PbE)
        out = binary_in.copy()
        out[ix] = np.mod(out[ix] + 1, 2)   # flip error bits
        return out
```

```
src = Source("The quick brown fox jumps over the lazy dog 0123456789")
ch = Channel(0.01)
noisy = ch.transmit(src.generate())
rx = Receiver(noisy, noisy)
print(f'BER: {rx.BERchan(src.generate()):0.4f}')
print(rx.received_text(src.LSBfirst))
```

```
BER: 0.0023
The quick brown fox jumps over the lazy dog 0523456789
```

Encoder Class

Create:

```
class Encoder:
```

Internal state:

```
G    # Encoder matrix
```

Methods:

```
encode()
```

```
class Encoder:

    def __init__(self, G):
        self.G = np.array(G, int)
        self.k, self.n = np.shape(self.G)

    def encode(self, src):
        N = int(np.ceil(len(src)/self.k))
        src_pad = np.zeros(N*self.k, int)
        src_pad[:len(src)] = src
        u = np.reshape(src_pad, (-1, self.k))   # split src string into blocks of length k
        B = np.mod(u@self.G, 2)
        return np.reshape(B, (1, -1)).flatten()
```

```
src2 = Source('Hi!')
G = [[1,0,0,0,1,1,1],[0,1,0,0,1,1,0],[0,0,1,0,1,0,1],[0,0,0,1,0,1,1]]
enc = Encoder(G)
coded = enc.encode(src2.generate())
print(coded)
```

```
[0 0 0 1 0 1 1 0 0 1 0 1 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 1 0 1
 0 0 1 1 0]
```

Decoder Class

Create:

```
class Decoder:
```

Internal state:

```
H    # Parity check matrix
```

Methods:

```
decode()
syndrome()
```

```
class Decoder:

    def __init__(self, H):
        self.H = np.array(H, int)
        self.nk, self.n = np.shape(self.H)
        self.map = self.__syn2bit()

    def __syn2bit(self):
        syn = self.H.T@np.array(2**np.arange(self.nk-1,-1,-1))  # syndrome in decimal
        map = -np.ones(self.n+1, int)
        for i in range(self.n):
            map[syn[i]] = i
        return map

    def syndrome(self, noisy):
        N = int(np.ceil(len(noisy)/self.n))
```

```
        noisy_pad = np.zeros(N*self.n, int)
        noisy_pad[:len(noisy)] = np.array(noisy, int)
        v = np.reshape(noisy_pad, (-1, self.n))   # split noisy string into blocks of length n
        return np.mod(v@self.H.T, 2)

    def decode(self, noisy):
        N = int(np.ceil(len(noisy)/self.n))
        noisy_pad = np.zeros(N*self.n, int)
        noisy_pad[:len(noisy)] = np.array(noisy, int)
        S = self.syndrome(noisy_pad)
        ss = S@np.array(2**np.arange(self.nk-1,-1,-1))
        print(ss)
        ixe = []
        for i in range(len(ss)):
            if ss[i]>0:
                ixe.append(self.n*i + self.map[ss[i]])
        print(ixe)
        corrected = np.copy(noisy_pad)
        corrected[ixe] = np.mod(corrected[ixe] + 1, 2)
        D = np.reshape(corrected, (-1, self.n))
        decoded = np.reshape(D[:,:self.n-self.nk], (1, -1)).flatten()
        return decoded


DD = np.array([[1,2,3],[4,5,6]])
print(DD)
print(DD[:,:2])
```

```
[[1 2 3]
 [4 5 6]]
[[1 2]
 [4 5]]
```

```
ch2 = Channel(0.05)
noisy = ch2.transmit(coded)
H = [[1,1,1,0,1,0,0],[1,1,0,1,0,1,0],[1,0,1,1,0,0,1]]
#H = [[1,1,1,1,1,1,1,0,0,0,0,1,0,0,0],
#     [1,1,1,1,0,0,0,1,1,1,0,0,1,0,0],
#     [1,1,0,0,1,1,0,1,1,0,1,0,0,1,0],
#     [1,0,1,0,1,0,1,1,0,1,1,0,0,0,1]]
dec = Decoder(H)
syn = dec.syndrome(noisy)
print(syn)
```

```
[[0 0 1]
 [0 0 0]
 [0 1 1]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
```

```
decoded2 = dec.decode(noisy)
rx2 = Receiver(noisy, decoded2)
print(coded)
print(noisy)
print(src2.generate())
print(decoded2)
#print(dec.map)
```

```
[1 0 3 0 0 0]
```

```
[np.int64(6), np.int64(17)]
[0 0 0 1 0 1 1 0 0 1 0 1 0 1 1 0 0 1 1 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 1 0 1
 0 0 1 1 0]
[1 0 1 1 0 0 0 0 0 1 0 1 0 1 1 0 0 0 1 0 0 0 1 1 0 0 1 1 1 0 0 0 1 1 1 0 1
 0 0 1 1 0]
[0 0 0 1 0 0 1 0 1 0 0 1 0 1 1 0 1 0 0 0 0 1 0 0]
[1 0 1 1 0 0 1 0 1 0 0 1 0 1 1 0 1 0 0 0 0 1 0 0]
```

```
print(f'BERchan: {rx2.BERchan(coded):0.4f}')
print(f'BERinfo: {rx2.BERinfo(src2.generate()):0.4f}')
print(rx2.received_text(src2.LSBfirst))
```

```
BERchan: 0.1190
BERinfo: 0.0833
Mi!
```